Pipeline report

Pipeline report 1
Cel Badania i Architektura Potoku (Pipeline) 1
Zestawienie Pomiarów Wydajnościowych 1
Wykresy Analizy Potoku i Skalowania 1
Bezpośrednie Odpowiedzi na Pytania Badawcze 2
Czas działania programu w funkcji M i K: 2
Wyznaczenie K_opt dla M=1M na badanej maszynie: 2
Wpływ dodania większej liczby wątków po stronie czytania oraz liczenia hash 3
Analiza Techniczna, Wykryte Problemy i Sugestie Poprawek 3
Analiza Wąskich Gardeł (Bottlenecks) 3
Możliwe Problemy i Sugestie Poprawek w Kodzie (Possible Fixes) 3
Środowisko testowe 3

Cel Badania i Architektura Potoku (Pipeline)

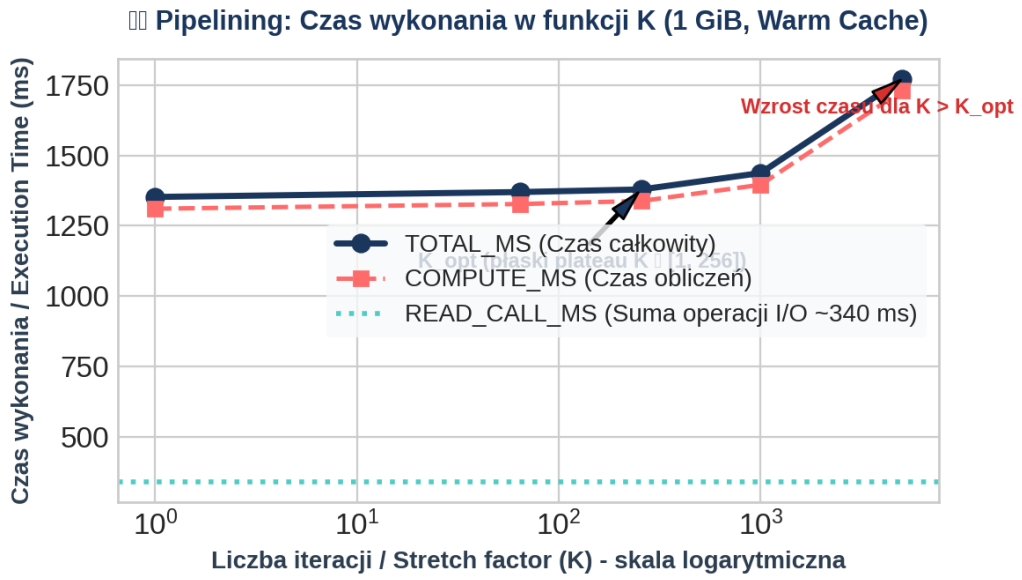
Celem niniejszej części badania jest analiza wielowątkowej architektury potokowej, która rozdziela operacje odczytu I/O z dysku oraz obliczenia funkcji skrótu. Aplikacja wykorzystuje ograniczony kanał synchronizacyjny, dzięki czemu czas wykonania programu zmienia się z relacji sumarycznej $T = T_{read} + T_{compute}$ na relację maksymalną: $T_{pipeline} = \max(T_{read}, T_{compute})$

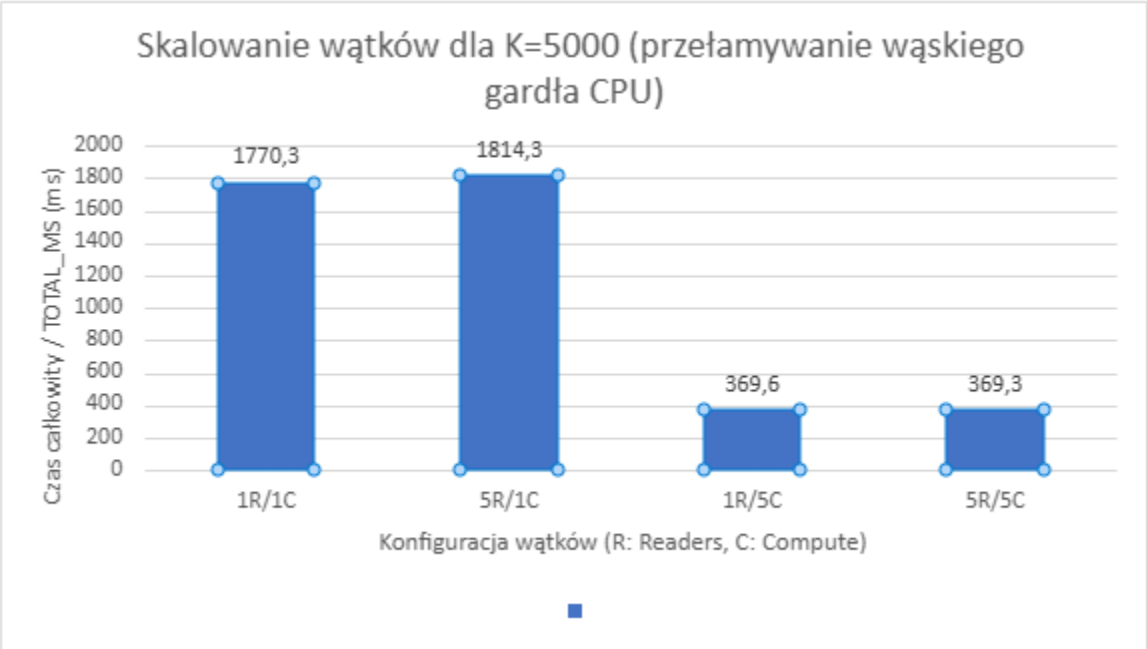
Zestawienie Pomiarów Wydajnościowych

K	Wątki (R/C)	Rozmiar Pliku	READ_CALL_MS	COMPUTE_MS	COMPUTE_WALL_MS	TOTAL_MS	Wąskie Gardło
1	1R / 1C	1 GiB (Warm)	329.4 ms	1310.6 ms	1352.3 ms	1352.4 ms	CPU (Compute)
64	1R / 1C	1 GiB (Warm)	341.7 ms	1315.1 ms	1357.4 ms	1357.6 ms	CPU (Compute)
256	1R / 1C	1 GiB (Warm)	339.5 ms	1339.0 ms	1380.6 ms	1380.8 ms	CPU (Compute)
1000	1R / 1C	1 GiB (Warm)	341.5 ms	1395.3 ms	1437.2 ms	1437.3 ms	CPU (Degradacja)
5000	1R / 1C	1 GiB (Warm)	334.6 ms	1722.2 ms	1762.6 ms	1762.7 ms	CPU (Ciężkie K)
5000	5R / 1C	1 GiB (Warm)	346.5 ms	1755.4 ms	1796.5 ms	1796.6 ms	CPU (Rywalizacja)
5000	1R / 5C	1 GiB (Warm)	343.2 ms	1783.0 ms	369.4 ms	369.5 ms	I/O (Dysk/RAM)
5000	5R / 5C	1 GiB (Warm)	396.1 ms	1788.9 ms	369.8 ms	370.0 ms	I/O (Overhead)
5000	1R / 1C	4 GiB (Cold)	1405.5 ms	7160.8 ms	7329.9 ms	7330.0 ms	CPU
1	1R / 1C	40 GiB (Cold)	12293.6 ms	52991.8 ms	54729.0 ms	54729.1 ms	CPU (Brak cache)

W powyższym zestawieniu przedstawiono czasy odczytu (READ_CALL_MS) oraz obliczeń (COMPUTE_MS). A także dodatkowe fazy orientacyjne: COMPUTE_WALL_MS oraz TOTAL_MS. Szczegółowe rozmieszczenie faz i zakres operacji można zweryfikować bezpośrednio w kodzie źródłowym programu.

Wykresy Analizy Potoku i Skalowania





Bezpośrednie Odpowiedzi na Pytania Badawcze

Czas działania programu w funkcji M i K: Czas wykonania T_TOTAL skaluje się liniowo w funkcji rozmiaru pliku (M). Natomiast w funkcji K występuje stabilny płaskowyż (plateau) w zakresie [1, K_opt], po czym dla K > K_opt następuje liniowy wzrost czasu z powodu przewagi kosztów obliczeniowych CPU nad operacjami I/O.

Wyznaczenie K_opt dla M=1M na badanej maszynie: Wartość optymalna wynosi K_opt ≈ 256. W zakresie K ∈ [1, 256] czas wykonania wynosi stałe ~1350–1379 ms (wąskim gardłem dla 1 wątku jest CPU). Zwiększenie K do 1000 wydłuża czas do 1437 ms, a przy K = 5000 aż do 1770 ms. W przypadku pliku ciągle transmitowanego z dysku (Cold Cache / 40 GiB) czas I/O (READ_CALL_MS) wzrasta proporcjonalnie do nośnika, jednak to 1 wątek obliczeniowy nadal stanowi główne wąskie gardło.

Wpływ dodania większej liczby wątków po stronie czytania oraz liczenia hash**:** Dodanie wątków czytających (5R / 1C) NIE daje żadnego przyspieszenia (czas wzrasta z 1762 ms do 1814 ms przez narzut synchronizacji). Natomiast dodanie wątków obliczeniowych (1R / 5C) drastycznie skraca czas całkowity z 1762.7 ms do 369.5 ms (4.77x przyspieszenia!), w pełni równoważąc potok do poziomu fizycznej przepustowości I/O (~343 ms)!

Analiza Techniczna, Wykryte Problamy i Sugestie Poprawek

Analiza Wąskich Gardeł (Bottlenecks)

Analiza logów ujawnia kluczowe zjawiska w architekturze potoku:

- **Single-Threaded Compute Bottleneck:** Dla pojedynczego wątku obliczeniowego (1C) liczenie funkcji stretch_md5 (nawet przy K=1) trwa ok. 1310 ms, podczas gdy odczyt 1 GiB z RAM/Cache trwa zaledwie 340 ms. Oznacza to, że kanał sync_channel jest stale pełny, a wątek czytający blakuje się na operacji send().
- **Przełamanie potoku przez sekwencjonowanie Mutex:** W dostarczonym kodzie wątki obliczeniowe dzielą kanał odbiorczy mpsc::Receiver poprzez Arc<Mutex<mpsc::Receiver>>. Gdy używamy 5 wątków (5C), blokada Mutex::lock() jest zwalniana przed obliczeniem hasha, co umożliwia idealne, zrównoważone przetwarzanie równoległe na procesorze Intel Core Ultra 7 255HX (20 rdzeni).

Możliwe Problemy i Sugestie Poprawek w Kodzie (Possible Fixes)

- **Poprawka 1: Usunięcie Mutex z kanału odbiorczego (Lock-Free Receiver Sharing):** Zamiast chronić pojedynczy odbiornik mpsc::sync_channel strukturą Arc<Mutex<...>>, lepszym rozwiązaniem w języku Rust jest zastosowanie biblioteki crossbeam-channel lub wieloodbiornikowego kanału spmc / mpmc. Wyeliminowałoby to narzut rywalizacji o blokadę (lock contention) przy wywołaniach recv().
- **Poprawka 2: Zwiększenie CHANNEL_CAPACITY z 2 do 8-16:** Pojemność kanału CHANNEL_CAPACITY = 2 jest bardzo mała. W przypadku chwilowych skoków opóźnień I/O (np. przy twardym błędzie strony w Cold Cache) wątek czytający natychmiast się blokuje. Zwiększenie bufora do 8-16 bloków (8-16 MiB RAM) wygładza wahania i zapewnia płynny przepływ danych.

- **Poprawka 3: Zero-Copy Buffer Reuse (Alokacja Pamięci):** W obecnym kodzie pętla czytająca alokuje nowy wektor vec![0u8; bytes_to_read] dla każdego bloku. Przy 40 960 blokach (dla pliku 40 GiB) powoduje to ponad 40 tysięcy alokacji pamięci na stercie (heap allocation). Zastosowanie puli buforów (buffer pool / recycling) całkowicie wyeliminowałoby narzut alokatora pamięci.

Środowisko testowe

Pomiary benchmarkowe zostały przeprowadzone na komputerze wyposażonym w procesor Intel Core Ultra 7 255HX, 32 GB pamięci RAM oraz 64-bitowy system operacyjny Windows. Program napisano w języku Rust i skompilowano w profilu produkcyjnym (--release). Badania wykonano przy użyciu wewnętrznego dysku SSD, uwzględniając zarówno warunki pamięci podręcznej systemu operacyjnego (Warm Cache), jak i pełne czyszczenie bufora pamięci RAM (Cold Cache) przy użyciu narzędzia RAMMap